

Neural Networks Practice: From Weighted Sums to Tiny Networks

This notebook introduces neural networks slowly.

Important idea:

A neuron starts with the same weighted-sum idea as regression, then applies an activation.

Today we will **not train** a neural network yet. We will build tiny neurons by hand so the pieces make sense before training them next time.

```
import matplotlib.pyplot as plt
```

Part 1 - Weighted sum

A model can combine multiple inputs:

```
score = w1*x1 + w2*x2 + w3*x3 + b
```

For delivery lateness, possible inputs are:

- distance
- traffic level
- rain indicator

Each input gets a weight.

```
def weighted_sum(inputs, weights, bias):  
    """Compute  $w_1x_1 + w_2x_2 + \dots + b$ ."""  
    # TODO 1: start total at bias  
    # TODO 2: loop through inputs and weights together using zip  
    # TODO 3: add  $x * w$  to total  
    # TODO 4: return total  
    pass  
  
inputs = [5.0, 0.8, 1.0]      # distance, traffic, rain  
weights = [3.0, 12.0, 5.0]  
bias = 2.0  
  
print(weighted_sum(inputs, weights, bias))
```

Part 2 - Activation

A weighted sum gives a raw score. For a yes/no decision, we need to convert the score into an output.

A simple activation is a threshold:

```
if score >= threshold: output = 1
else: output = 0
```

For delivery, output 1 can mean "late" and output 0 can mean "not late".

```
def step_activation(score, threshold):
    """Return 1 if score is at least threshold, otherwise 0."""
    # TODO 5: implement the threshold rule
    pass

print(step_activation(32, threshold=30))
print(step_activation(22, threshold=30))
```

Part 3 - One neuron

A neuron is two steps:

1. compute weighted sum
2. apply activation

```
def neuron(inputs, weights, bias, threshold):
    """A tiny neuron: weighted sum followed by activation."""
    # TODO 6: compute score using weighted_sum
    # TODO 7: return step_activation(score, threshold)
    pass

examples = [
    ([2.0, 0.1, 0.0], "short clear delivery"),
    ([6.0, 0.9, 1.0], "long rainy traffic delivery"),
]

for inputs, label in examples:
    prediction = neuron(inputs, weights=[3.0, 12.0, 5.0], bias=2.0,
                        threshold=30)
    print(label, "-> late?", prediction)
```

Part 4 - Visual intuition: one neuron is one simple boundary

A single neuron often acts like one simple rule. That can be useful, but it is limited.

```

# We will visualize a simple 2D late/not-late dataset.
points = [
    (1, 1, 0),
    (2, 2, 0),
    (3, 1, 0),
    (4, 5, 1),
    (5, 4, 1),
    (6, 6, 1),
]

xs0 = [x for x, y, label in points if label == 0]
ys0 = [y for x, y, label in points if label == 0]
xs1 = [x for x, y, label in points if label == 1]
ys1 = [y for x, y, label in points if label == 1]

plt.figure()
plt.scatter(xs0, ys0, label="not late")
plt.scatter(xs1, ys1, label="late")
plt.xlabel("distance")
plt.ylabel("traffic")
plt.title("A pattern one neuron could separate")
plt.legend()
plt.show()

```

Part 5 - Logic neurons: AND and OR

Now we use tiny binary inputs. This is not because real neural networks only do logic; it is because logic makes weights easy to reason about.

Truth table:

x1	x2	AND	OR
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

```

logic_inputs = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1],
]

# TODO 8: choose weights, bias, and threshold that create AND.
and_weights = [1, 1]
and_bias = 0
and_threshold = 2

```

```

print("AND neuron")
for x in logic_inputs:
    print(x, "->", neuron(x, and_weights, and_bias, and_threshold))

# TODO 9: choose weights, bias, and threshold that create OR.
or_weights = [1, 1]
or_bias = 0
or_threshold = 1

print("OR neuron")
for x in logic_inputs:
    print(x, "->", neuron(x, or_weights, or_bias, or_threshold))

```

Part 6 - Why one neuron is not enough: XOR

XOR means "exactly one input is 1".

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Try to imagine one straight line separating the 1s from the 0s. It does not work.

```

xor_points = [
    (0, 0, 0),
    (0, 1, 1),
    (1, 0, 1),
    (1, 1, 0),
]

xs0 = [x for x, y, label in xor_points if label == 0]
ys0 = [y for x, y, label in xor_points if label == 0]
xs1 = [x for x, y, label in xor_points if label == 1]
ys1 = [y for x, y, label in xor_points if label == 1]

plt.figure()
plt.scatter(xs0, ys0, label="0")
plt.scatter(xs1, ys1, label="1")
plt.xticks([0, 1])
plt.yticks([0, 1])
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("XOR pattern")
plt.legend()
plt.show()

```

Part 7 - A tiny network solves XOR

A network can combine simpler neurons.

One construction:

- Hidden neuron A: OR
- Hidden neuron B: NAND, meaning "not both"
- Output neuron: AND of those hidden outputs

So XOR becomes:

```
XOR = AND(OR(x1, x2), NAND(x1, x2))

def nand_neuron(x):
    """Return 1 unless both inputs are 1."""
    # TODO 10: choose weights/bias/threshold for NAND.
    # Hint: one simple method is weights [-1, -1], bias 2, threshold 1
    pass

def xor_network(x):
    """Tiny network made from OR, NAND, and AND."""
    # TODO 11: compute hidden1 = OR(x)
    # TODO 12: compute hidden2 = NAND(x)
    # TODO 13: feed [hidden1, hidden2] into the AND neuron
    pass

print("XOR network")
for x in logic_inputs:
    print(x, "->", xor_network(x))
```

Final concept check

Answer in comments:

1. What is the difference between a weighted sum and an activation?
2. Is using many inputs the same thing as using a neural network?
3. Why can a network solve some patterns one neuron cannot?
4. What will training do next week?

```
# TODO 14: Write short answers here.
# 1.
# 2.
# 3.
# 4.
```